Technical Requirement Document (TRD)

Overview

This document defines the technical requirements for a college-first rental marketplace focused on students, owners, and admins. The system is designed around a practical, stable architecture using Vite + React for the web frontend, React Native for mobile, FastAPI for backend APIs, Firebase Authentication for identity, Firestore for primary data storage, Cloudinary for media, and OpenStreetMap + Leaflet for map display.[cite:29] [cite:31] [cite:37]

The architecture should optimize for long-term maintainability, simple team onboarding, low operational overhead, and predictable scaling. The system should avoid unnecessary microservices, message buses, or heavy infrastructure until actual traffic and complexity justify them.[cite:40] [cite:43] [cite:47]

System architecture overview

High-level architecture

The platform should use a modular monolith architecture.

- **Web frontend**: Vite + React + Tailwind application for student browsing, owner dashboard, and admin panel.
- **Mobile frontend**: React Native app for student and owner usage.
- **Backend API**: FastAPI service exposing REST APIs for business logic, validation, moderation, search orchestration, and authorization checks.
- **Authentication**: Firebase Authentication for sign-in and user identity tokens.
- **Database**: Firestore as the primary operational database.
- **Media storage**: Cloudinary for image upload, optimization, and delivery.
- **Maps**: OpenStreetMap with Leaflet for listing and college map display.[cite:29] [cite:31] [cite:35] [cite:37]

This system should be deployed as a single backend service with clear internal modules rather than separate microservices. A layered architecture with API, service, and repository boundaries is a stable pattern for FastAPI applications and improves testability without unnecessary complexity.[cite:40]

Recommended backend layers

- **API layer**: route handlers, request parsing, response formatting.
- **Service layer**: business rules, workflow orchestration, validation.
- **Repository layer**: Firestore access abstraction and query wrappers.
- **Infrastructure layer**: Firebase Admin SDK, Cloudinary integration, analytics hooks, and shared utilities.[cite:40]

Frontend responsibilities

Shared frontend responsibilities

Both web and mobile clients should remain thin clients. They should focus on rendering, local interaction state, input capture, and API consumption rather than embedding business rules.

Frontend responsibilities should include:

- User authentication with Firebase client SDK.
- Token submission to backend APIs.
- Listing search UI and filter state.
- Listing details rendering.
- Form validation at UX level.
- Image upload initiation flow.
- Role-based navigation and route protection.
- Optimistic UI only where safe and reversible.

Business-critical validation must still be enforced on the backend because client-side checks are not trustworthy.[cite:42] [cite:45]

Web frontend responsibilities

The web app should provide:

- Public search and browsing experience.
- Listing detail pages.
- Student saved listings and alerts UI.
- Owner listing management dashboard.
- Full admin control panel.
- Analytics dashboards for admin.

Mobile app responsibilities

The React Native app should prioritize:

- Student search and listing discovery.
- Saved listings and recent views.
- Owner listing management essentials.
- Push-ready architecture later, but no notification system in V1.

Admin workflows should remain web-first in V1 to reduce mobile complexity.

Backend responsibilities

The FastAPI backend should own all critical business logic and cross-client consistency. [cite:40] [cite:43]

Primary backend responsibilities:

- Verify Firebase ID tokens on protected endpoints.[cite:42] [cite:45]
- Resolve authenticated user role and status.
- Enforce role-based access control for admin, student, and owner routes.[cite:42]
- Manage listing CRUD with approval workflow.
- Compute distance between listing coordinates and selected college coordinates.
- Validate required property-type-specific fields.
- Sanitize and normalize listing payloads.
- Control listing visibility and moderation state.
- Manage reviews, shortlists, inquiries, recent views, and alerts.
- Expose analytics aggregation endpoints for admin reporting.
- Integrate with Cloudinary upload flow.
- Log moderation and critical platform actions.

The backend should not rely on the frontend to determine who can approve listings, who can edit a property, or whether a listing is visible. Those rules must remain server controlled. [cite:42] [cite:45]

Database schema proposal

Firestore should be used with a collection-oriented model and selective denormalization. Firestore data modeling works best when documents are organized for direct read patterns rather than relational joins, and denormalization is commonly needed for query efficiency. [cite:41] [cite:44] [cite:47]

Collections

`users`

Stores all authenticated users.

Suggested fields:

- `uid`
- `role` (`admin`, `student`, `owner`)
- `name`
- `phone`
- `email`
- `photo_url`
- `status` (`active`, `blocked`, `pending_verification`)
- `created_at`
- `updated_at`
- `last_login_at`
- `college_id` (optional for students)
- `verification_state` (mainly for owners)

`colleges`

Stores searchable college master data.

Suggested fields:

- `college_id`
- `name`
- `short_name`
- `address`
- `city`
- `state`
- `latitude`
- `longitude`
- `status`
- `created_at`
- `updated_at`

`properties`

Core property listing documents.

Suggested common fields:

- `property_id`
- `owner_uid`
- `college_ids` (array if relevant to more than one nearby college; otherwise single primary `college_id` is simpler for V1)
- `primary_college_id`
- `property_type`
- `title`
- `description`
- `address_text`
- `latitude`
- `longitude`
- `rent_min`
- `rent_max`
- `security_deposit`
- `amenities` (array)
- `food_available`
- `food_menu`
- `rules`
- `availability_status` (`available`, `occupied`, `coming_soon`)
- `approval_status` (`pending`, `approved`, `rejected`)
- `visibility_status` (`live`, `hidden`, `archived`)
- `featured`
- `rating_avg`
- `rating_count`
- `review_count`
- `cover_image_url`
- `image_urls`
- `created_at`
- `updated_at`
- `published_at`

`property_room_options`

Recommended as a subcollection under `properties/{property_id}/room_options` if room options are variable. This avoids oversized root documents while keeping room data tightly scoped.

Fields:

- `option_id`
- `label` (`single`, `2-sharing`, `3-sharing`)
- `price`
- `deposit` (optional override)
- `available_count`
- `status`

`property_metadata`

Type-specific fields can either be embedded directly inside a `metadata` object in the property document or separated into typed sub-documents. For V1, a single `metadata` object is simpler and stable enough.

Examples:

- PG: `gender`, `mess_timing`, `gate_timing`, `warden_contact`
- Flat: `bhk_type`, `furnishing`, `floor`, `lift`, `society_name`
- Shop: `size_sqft`, `allowed_use_type`, `maintenance_charge`
- Hostel: `total_capacity`, `study_hall`, `curfew_time`, `warden_on_site`
- Single Room: `bathroom_type`, `kitchen_access`, `furnished`
- Co-living: `community_events`, `coworking`, `min_stay_duration`

`reviews`

Stores user reviews for properties.

Suggested fields:

- `review_id`
- `property_id`
- `user_uid`
- `rating`
- `title`
- `comment`
- `stay_status`

- `moderation_status`
- `created_at`

`inquiries`

Stores interest submissions.

Suggested fields:

- `inquiry_id`
- `property_id`
- `owner_uid`
- `student_uid` (nullable if guest inquiry is allowed)
- `name`
- `phone`
- `message`
- `source`
- `created_at`

`shortlists`

A per-user saved listing record.

Suggested fields:

- `user_uid`
- `property_id`
- `created_at`

This can also be modeled as a subcollection under user documents if access is always user-scoped.

`recent_views`

Tracks recently viewed listings for logged-in users.

Suggested fields:

- `user_uid`
- `property_id`
- `viewed_at`

`alerts`

Stores saved search alerts.

Suggested fields:

- `alert_id`
- `user_uid`
- `college_id`
- `property_type`
- `budget_min`
- `budget_max`
- `radius_km`
- `gender`
- `amenities`
- `active`
- `created_at`

`admin_logs`

Stores moderation and admin audit records.

Suggested fields:

- `log_id`
- `admin_uid`
- `action_type`
- `target_type`
- `target_id`
- `reason`
- `metadata`
- `created_at`

Firestore modeling notes

Firestore queries work best when documents are shaped around read patterns and indexed fields. Avoid trying to mimic fully relational joins in Firestore, and instead duplicate selected display data where it reduces repeated lookups.[cite:44] [cite:47]

Recommended practices:

- Keep documents reasonably small.
- Store display-friendly fields on listing documents to reduce joins.

- Use subcollections for variable child entities like room options or large review sets.
- Precompute aggregate fields such as `rating_avg`, `rating_count`, and listing completeness score.
- Create composite indexes only for actual query patterns.
- Avoid unbounded array growth in a single document.[cite:41] [cite:44] [cite:47]

API structure

The API should be REST-based and versioned under `/api/v1`. This is sufficient for V1 and easier to maintain than introducing GraphQL or event-driven interfaces.

Route groups

Auth and profile

- `GET /api/v1/me`
- `PATCH /api/v1/me`
- `POST /api/v1/me/complete-profile`

Colleges

- `GET /api/v1/colleges`
- `GET /api/v1/colleges/{college_id}`
- `POST /api/v1/admin/colleges`
- `PATCH /api/v1/admin/colleges/{college_id}`
- `DELETE /api/v1/admin/colleges/{college_id}`

Public property discovery

- `GET /api/v1/properties`
- `GET /api/v1/properties/{property_id}`
- `GET /api/v1/properties/search`

Typical query params:

- `college_id`
- `radius_km`
- `property_type`
- `gender`
- `budget_min`
- `budget_max`
- `amenities`

- `sort`
- `page`
- `limit`

Owner property management

- `POST /api/v1/owner/properties`
- `PATCH /api/v1/owner/properties/{property_id}`
- `DELETE /api/v1/owner/properties/{property_id}`
- `PATCH /api/v1/owner/properties/{property_id}/availability`
- `GET /api/v1/owner/properties`
- `GET /api/v1/owner/properties/{property_id}/inquiries`

Reviews and engagement

- `POST /api/v1/properties/{property_id}/reviews`
- `GET /api/v1/properties/{property_id}/reviews`
- `POST /api/v1/properties/{property_id}/shortlist`
- `DELETE /api/v1/properties/{property_id}/shortlist`
- `GET /api/v1/me/shortlists`
- `GET /api/v1/me/recent-views`
- `POST /api/v1/properties/{property_id}/inquiries`
- `POST /api/v1/me/alerts`
- `GET /api/v1/me/alerts`
- `PATCH /api/v1/me/alerts/{alert_id}`
- `DELETE /api/v1/me/alerts/{alert_id}`

Admin moderation

- `GET /api/v1/admin/listings/pending`
- `PATCH /api/v1/admin/properties/{property_id}/approve`
- `PATCH /api/v1/admin/properties/{property_id}/reject`
- `PATCH /api/v1/admin/properties/{property_id}/feature`
- `PATCH /api/v1/admin/properties/{property_id}/hide`
- `GET /api/v1/admin/users`
- `PATCH /api/v1/admin/users/{uid}/status`
- `GET /api/v1/admin/analytics/overview`
- `GET /api/v1/admin/logs`

API design principles

- Keep request and response shapes stable and typed with Pydantic models.[cite:40]
- Use pagination on listing and admin endpoints.
- Return machine-readable error codes in addition to messages.
- Enforce validation centrally.
- Keep controller logic thin and move orchestration to services.[cite:40]

Authentication strategy

Firebase Authentication should be the identity provider, while FastAPI should act as the authorization and resource server.[cite:42] [cite:45]

Authentication flow

1. User signs in on client using Firebase Authentication.
2. Client receives Firebase ID token.
3. Client sends `Authorization: Bearer <token>` to backend.
4. Backend verifies the token using Firebase Admin SDK.[cite:42] [cite:45]
5. Backend loads the user profile and role from Firestore.
6. Backend authorizes route access based on role, status, and resource ownership.

Role model

- `admin`
- `student`
- `owner`

The backend should not trust a role value supplied by the client body. Role resolution should come from trusted backend-side user records or verified custom claims.[cite:42]

Authorization rules

- Public listing search and detail endpoints may be anonymous.
- Shortlists, alerts, reviews, and profile actions require authenticated student access.
- Listing creation and editing require authenticated owner access.
- Moderation, college management, and analytics require admin access.
- Owners may only modify their own properties.
- Admin may override visibility and status of any property.

Security notes

- Do not log raw ID tokens or personal secrets.[cite:40]
- Validate token expiration and revocation handling where needed.[cite:42] [cite:48]
- Use HTTPS only in all environments.
- Maintain centralized permission checks rather than scattered route logic.

Third-party dependencies

Required dependencies

- **Firebase Authentication** for sign-in and identity management.[cite:30] [cite:42]
- **Firebase Admin SDK** for backend token verification and trusted Firebase access.[cite:42] [cite:45]
- **Cloud Firestore** as the operational database.[cite:37] [cite:47]
- **Cloudinary** for media upload and transformation.
- **OpenStreetMap** for map data and **Leaflet** for map rendering.[cite:29] [cite:35]
- **Firebase Analytics / Google Analytics** for free event and usage tracking.[cite:30] [cite:36]

Frontend dependencies

- React
- Tailwind CSS
- React Router
- React Query or equivalent API-state library
- Form library such as React Hook Form
- Zod or schema validation library for client-side validation
- Leaflet bindings for React where needed

Backend dependencies

- FastAPI
- Uvicorn or Gunicorn + Uvicorn workers
- Pydantic Settings for configuration validation.[cite:40]
- Firebase Admin SDK
- Cloudinary SDK
- Structured logging library

These dependencies are sufficient for a stable V1. Avoid adding Redis, Celery, Kafka, GraphQL, or Elasticsearch in the first version unless clear production pain appears.[cite:40] [cite:43]

Scalability considerations

The goal is not to maximize theoretical scale on day one. The goal is to keep the architecture simple while avoiding future dead ends.

Application scalability

- Use modular monolith boundaries so modules can later be separated if truly needed. [cite:40]
- Keep services stateless so the API can be horizontally scaled behind a load balancer.
- Avoid local filesystem dependency for uploads because Cloudinary already externalizes media storage.
- Use structured logs and health endpoints from the beginning.[cite:40]

Firestore scalability

- Design queries around indexed access patterns.[cite:47]
- Precompute aggregates such as rating summaries and listing counters.[cite:44]
- Avoid large fan-out writes where unnecessary.
- Denormalize selectively for high-read listing screens.[cite:44]
- Archive stale listings rather than keeping every state in hot queries.

Search scalability

For V1, Firestore-backed filtered listing search is acceptable if filters are kept practical. If search volume, compound filters, or ranking complexity grows significantly, a search engine such as Algolia or OpenSearch can be introduced later. That should be an evolution step, not a starting dependency.

Analytics scalability

Use event tracking from the start but keep dashboards limited to high-value operational metrics. If admin reporting becomes too expensive to compute on demand, add scheduled aggregation jobs before adding a data warehouse.

Future-safe decisions

The following decisions improve long-term stability without overengineering:

- Strong API versioning from day one.
- Clear domain modules for users, colleges, properties, reviews, moderation, and analytics.
- Centralized auth middleware.
- Typed request and response contracts.
- Separate approval and visibility states on listings.
- Audit logging for admin operations.

- Cloud-based media handling instead of local uploads.[cite:40] [cite:42] [cite:47]

Operational recommendations

- Start with one production backend deployment and one staging deployment.
- Add automated tests for auth, listing moderation, and search filtering first.[cite:40]
- Keep environment configuration explicit and validated.[cite:40]
- Monitor error rates, response times, and failed token verifications.
- Document Firestore indexes as part of deployment setup.

Final recommendation

A modular monolith using FastAPI, Firebase Auth, Firestore, Cloudinary, and OpenStreetMap is the right technical shape for this product. It is stable enough for long-term growth, simple enough for a small team to ship, and flexible enough to evolve later without forcing early infrastructure complexity.[cite:29] [cite:31] [cite:40] [cite:47]